

INSTITUTO PROFESIONAL
SAN SEBASTIAN

EVALUACIÓN UNIDAD 1

DESARROLLO DE APLICACIONES MÓVILES

APLICACIÓN MÓVIL DE TAREAS CON IONIC

ALUMNO

JOHHAN URRUTIA LEIVA

DOCENTE

VICENTE ZAPATA

SECCIÓN

50

CARRERA

ING. EN INFORMÁTICA

REPOSITORIO GIT

https://github.com/JohhanUrrutia/todolistapp-ionic-evu1-urrutia-johhan

Introducción

Objetivo por desarrollar

Este informe tiene por objetivo demostrar detalladamente el desarrollo de la evaluación de la unidad uno del ramo Desarrollo de aplicaciones móviles. Como principal consigna se solicita crear una aplicación en la que se puedan registrar tareas, y con ello poder eliminarlas, mediante un formulario el cual permita recibir toda la información requerida.

Proceso de desarrollo

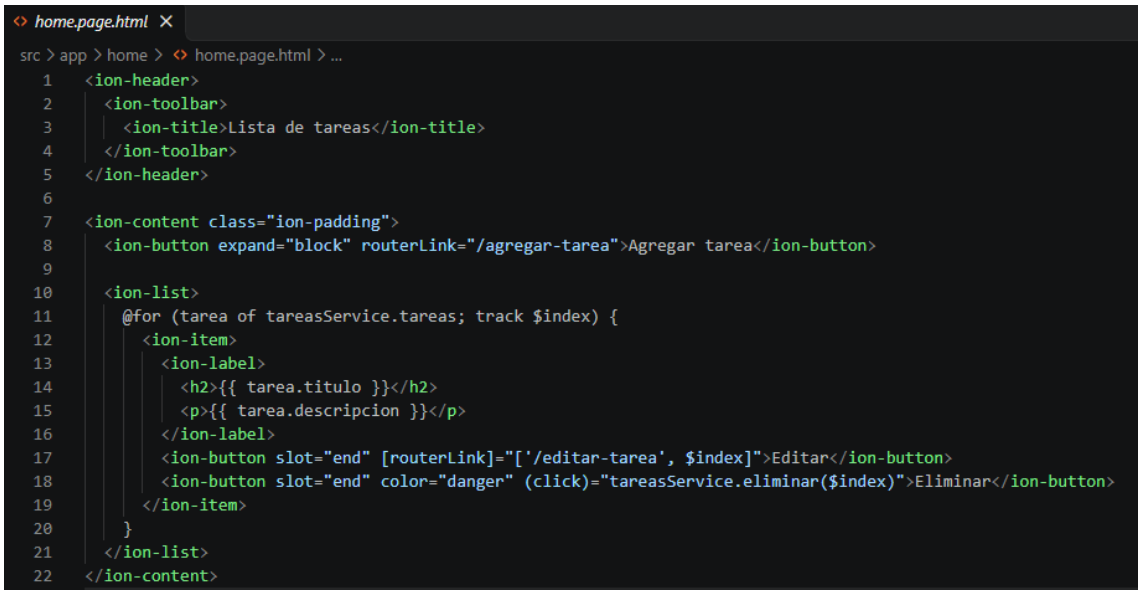
Desarrollo con Ionic

Ruta de archivo: src/app/services/tareas.service.ts.



Como base del proyecto, se creó un servicio llamado ‘TareasService’, el cual se encarga de almacenar la lista de tareas y compartirla entre ambas interfaces. Además, se definió la interfaz ‘Tarea’, la cual representa a cada tarea con su título y descripción. Dentro del servicio tenemos tres métodos: ‘agregar’, que añade una nueva tarea a la lista; ‘editar’, que reemplaza una tarea según su posición; y ‘eliminar’, que quita la tarea de la lista, es decir, toda la lógica de las tareas se concentra en un solo archivo.

Ruta de archivo: src/app/home/home.page.html.



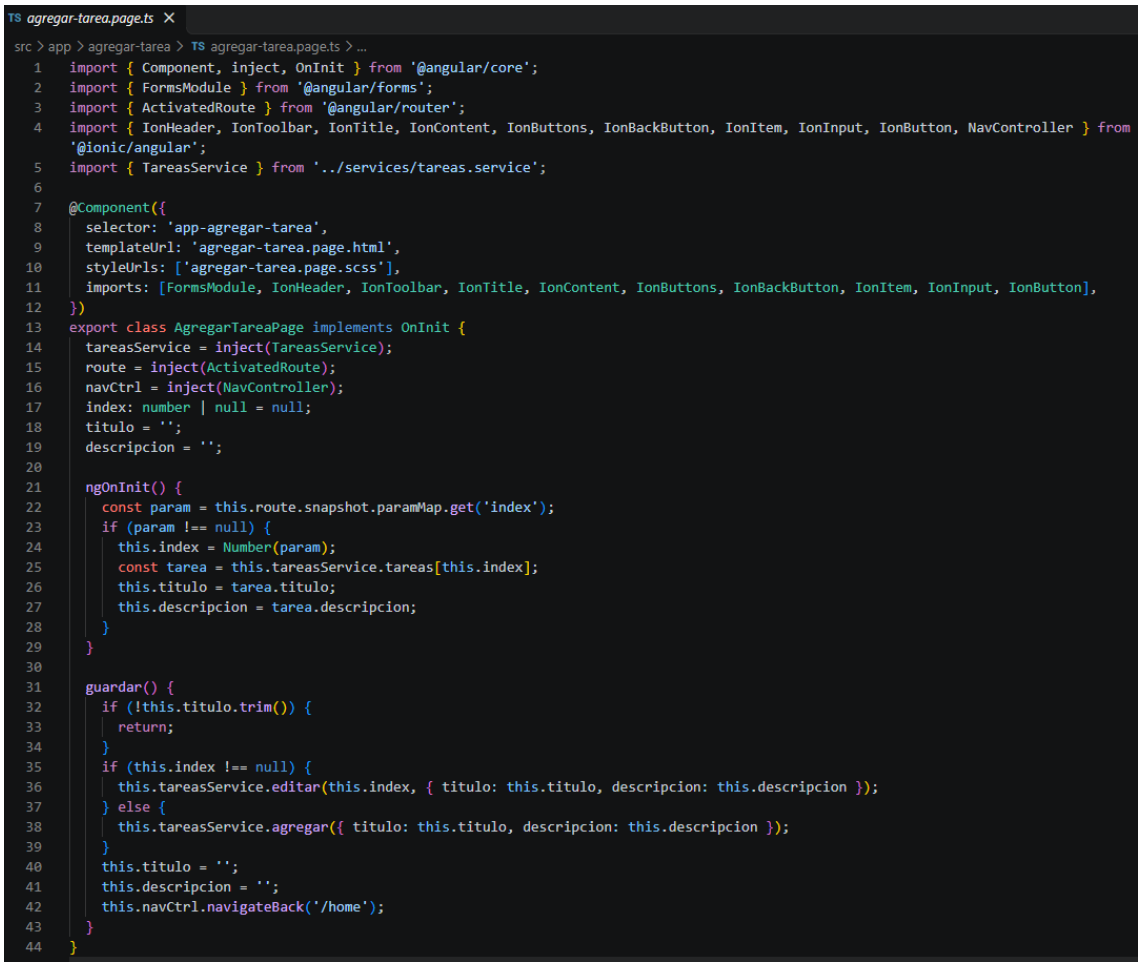
```
home.page.html
src > app > home > home.page.html > ...
<ion-header>
  <ion-toolbar>
    <ion-title>Lista de tareas</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <ion-button expand="block" routerLink="/agregar-tarea">Agregar tarea</ion-button>

  <ion-list>
    @for (tarea of tareasService.tareas; track $index) {
      <ion-item>
        <ion-label>
          <h2>{{ tarea.titulo }}</h2>
          <p>{{ tarea.descripcion }}</p>
        </ion-label>
        <ion-button slot="end" [routerLink]="['/editar-tarea', $index]">Editar</ion-button>
        <ion-button slot="end" color="danger" (click)="tareasService.eliminar($index)">Eliminar</ion-button>
      </ion-item>
    }
  </ion-list>
</ion-content>
```

En la interfaz principal, se utilizó un ciclo ‘@for’ para recorrer la lista de tareas almacenada en el servicio. Por cada tarea se genera un ‘ion-item’, el cual muestra el título y la descripción mediante un ‘ion-label’. Además, cada ‘ion-item’ contiene dos botones: ‘Editar’, que nos dirige al formulario enviando la posición de la tarea, y ‘Eliminar’, que llama al método del servicio para quitarla de la lista.

Ruta de archivo: src/app/agregar-tarea/agregar-tarea.page.ts.



```ts
import { Component, inject, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { ActivatedRoute } from '@angular/router';
import { IonHeader, IonToolbar, IonTitle, IonContent, IonButtons, IonBackButton, IonItem, IonInput, IonButton, NavController } from '@ionic/angular';
import { TareasService } from '../services/tareas.service';

@Component({
  selector: 'app-agregar-tarea',
  templateUrl: 'agregar-tarea.page.html',
  styleUrls: ['agregar-tarea.page.scss'],
  imports: [FormsModule, IonHeader, IonToolbar, IonTitle, IonContent, IonButtons, IonBackButton, IonItem, IonInput, IonButton],
})
export class AgregarTareaPage implements OnInit {
  tareasService = inject(TareasService);
  route = inject(ActivatedRoute);
  navCtrl = inject(NavController);
  index: number | null = null;
  titulo = '';
  descripcion = '';

  ngOnInit() {
    const param = this.route.snapshot.paramMap.get('index');
    if (param !== null) {
      this.index = Number(param);
      const tarea = this.tareasService.tareas[this.index];
      this.titulo = tarea.titulo;
      this.descripcion = tarea.descripcion;
    }
  }

  guardar() {
    if (!this.titulo.trim()) {
      return;
    }
    if (this.index !== null) {
      this.tareasService.editar(this.index, { titulo: this.titulo, descripcion: this.descripcion });
    } else {
      this.tareasService.agregar({ titulo: this.titulo, descripcion: this.descripcion });
    }
    this.titulo = '';
    this.descripcion = '';
    this.navCtrl.navigateBack('/home');
  }
}
```

Para el formulario, se reutilizó la misma página tanto para agregar como para editar una tarea. Al iniciar la página, el método ‘ngOnInit’ verifica si es que se recibió la posición de una tarea en la ruta; si es así, carga su título y descripción en los campos para poder editarlos. Por consiguiente, el método ‘guardar’ valida que el título no esté vacío y, dependiendo del caso, edita o agrega la tarea. Finalmente, se limpian los campos y la aplicación nos devuelve a la interfaz principal.

Ruta de archivo: src/app/agregar-tarea/agregar-tarea.page.html.

agregar-tarea.page.html

src > app > agregar-tarea > agregar-tarea.page.html > ...

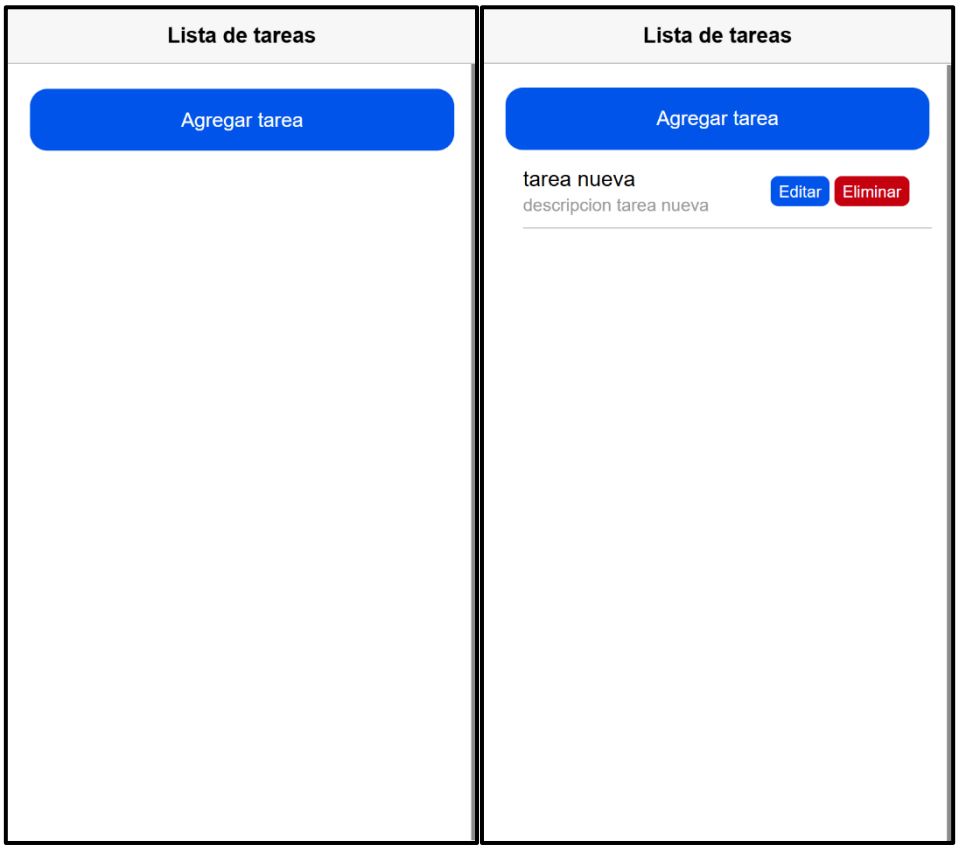
```
<ion-header>
  <ion-toolbar>
    <ion-buttons slot="start">
      <ion-back-button defaultHref="/home"></ion-back-button>
    </ion-buttons>
    <ion-title>{{ index !== null ? 'Editar tarea' : 'Agregar tarea' }}</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <ion-item>
    <ion-input label="Título" labelPlacement="floating" [(ngModel)]="titulo"></ion-input>
  </ion-item>
  <ion-item>
    <ion-input label="Descripción" labelPlacement="floating" [(ngModel)]="descripcion"></ion-input>
  </ion-item>
  <ion-button expand="block" (click)="guardar()">Guardar</ion-button>
</ion-content>
```

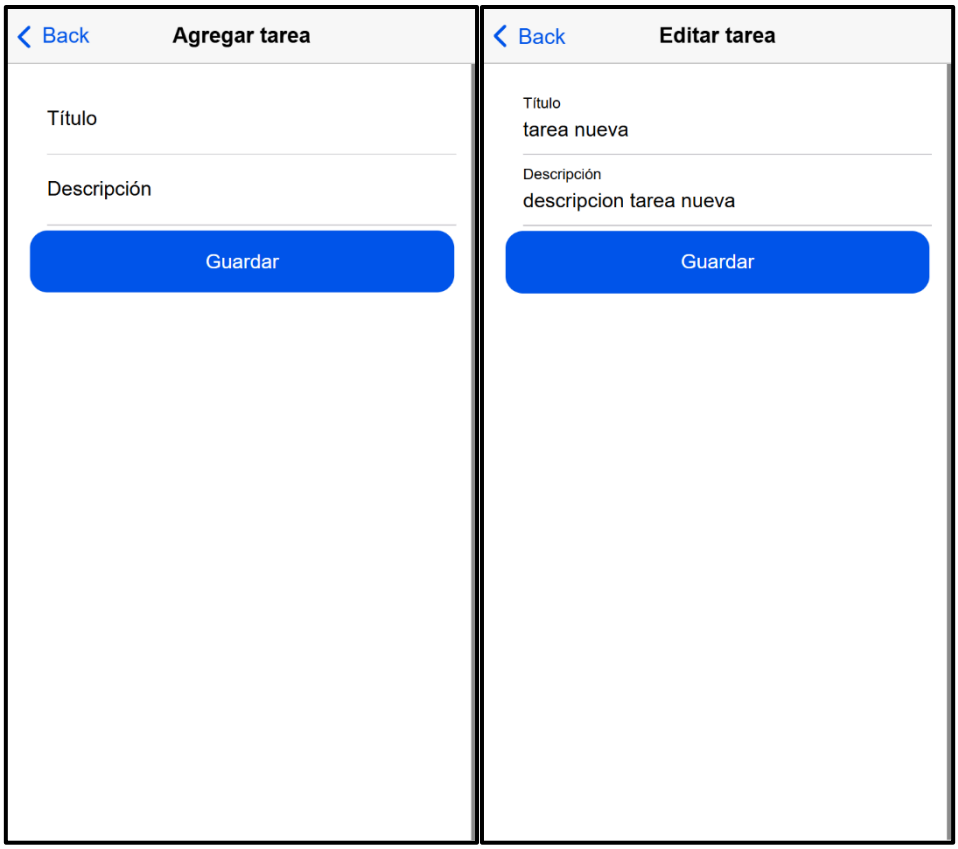
En el formulario se utilizaron dos ‘ion-input’, uno para el título y otro para la descripción de la tarea. Cada campo está enlazado mediante ‘[(ngModel)]’ a una variable del componente, vale decir, lo que escribe el usuario se guarda de forma automática en el código. Para guardar la tarea, se utilizó un ‘ion-button’, el cual ejecuta el método ‘guardar()’ explicado anteriormente.

Aplicación en funcionamiento

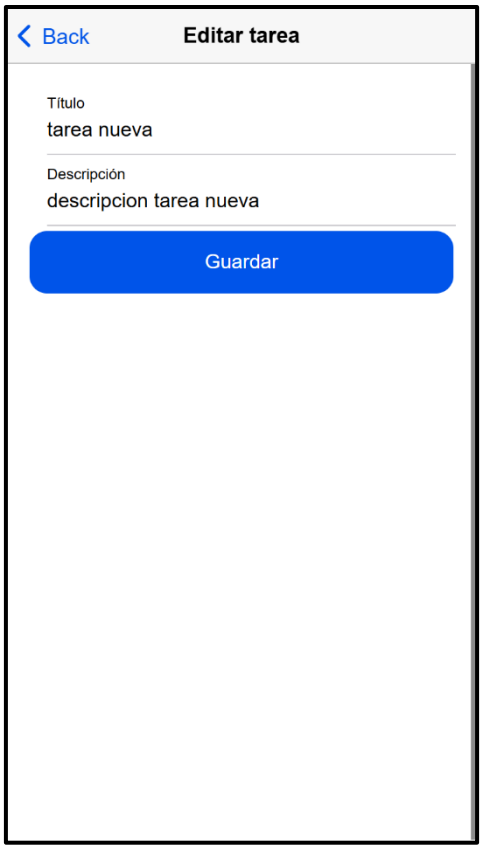
Registro de tareas



Al ingresar a la aplicación, como interfaz principal tenemos la página ‘Lista de tareas’. En la parte superior se encuentra el ion-header con su título, y en el ion-content se muestra el botón ‘Agregar tarea’ junto con la lista de tareas registradas. Cada tarea está representada por un ion-item, el cual contiene su título y su descripción, es decir, cada registro se muestra de forma individual dentro de la lista. Además, cada ion-item tiene dos botones: ‘Editar’, el cual nos permitirá modificar la tarea, y ‘Eliminar’, el cual quita esa tarea de la lista sin afectar a las demás.



Al presionar el botón ‘Agregar tarea’, la aplicación nos dirige a una nueva interfaz en donde se encuentra el formulario. Este formulario está compuesto por dos campos ion-input, uno para el título y otro para la descripción de la tarea. Para guardar la tarea, se utiliza un ion-button llamado ‘Guardar’, el cual agrega la tarea a la lista y con ello limpia los campos, vale decir, el formulario queda vacío para la próxima vez que se utilice. Finalmente, la aplicación nos devuelve de forma automática a la página principal, en donde podemos ver la tarea recién agregada.



Al presionar el botón ‘Editar’ de una tarea, la aplicación nos dirige al formulario, pero esta vez con el título ‘Editar tarea’ en la parte superior. Como podemos ver, los campos ion-input ya vienen cargados con el título y la descripción de la tarea seleccionada, es decir, no es necesario volver a escribir toda la información. Al realizar los cambios y presionar el botón ‘Guardar’, la tarea se actualiza en la lista, se limpian los campos y la aplicación nos devuelve a la interfaz principal. Como información adicional, en la parte superior izquierda se encuentra una flecha para volver a la página principal sin guardar los cambios.

Conclusión

Al finalizar este proyecto, podemos ver que fue posible desarrollar una aplicación de lista de tareas funcional y con un código simple. Para ello, se separó cada interfaz en su propia carpeta y se creó un servicio en donde se concentra toda la lógica de las tareas, es decir, cada parte del proyecto cumple una función específica sin afectar a las demás. De esta manera, la aplicación nos permite agregar, editar y eliminar tareas de forma ordenada, y con ello mantener una navegación clara entre la interfaz principal y el formulario.

En cuanto al uso de Ionic, podemos concluir que es una herramienta sumamente útil para este tipo de proyectos. Un claro ejemplo son sus componentes, como ‘ion-header’, ‘ion-content’, ‘ion-item’, ‘ion-input’ e ‘ion-button’, los cuales ya vienen con un diseño adaptado a dispositivos móviles, vale decir, no es necesario construir la interfaz desde cero. Además, al trabajar en conjunto con Angular, nos permite organizar el código en páginas, rutas y servicios de manera sencilla. Por consiguiente, Ionic se caracteriza por ser una herramienta ágil y fácil de usar, ideal para desarrollar aplicaciones móviles con una sola base de código.